Theoretical Manual

Contents

This manual presents the mathematical and algorithmic foundations underpinning each subsystem of the CourtSweeper. It formalises the sensor models, kinematic equations, control laws, navigation algorithms, communication protocols, and actuation principles.

1. Perception Models

1.1. Camera Pinhole Model

The system adopts the standard pinhole camera model. Let the camera intrinsic matrix be:

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad P = \begin{pmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{1}$$

For the CourtSweeper, the intrinsic parameters are approximated as $f_x = f_y = W$ and $(c_x, c_y) = (\frac{W}{2}, \frac{H}{2})$, where $W \times H$ is the resized frame dimension (typically 640×360). The lens is assumed to be distortion-free (Brown-Conrady model with all $k_i = 0$).

Table 1: Camera Intrinsic Parameters

Parameter	Value	Min.	Typical	Max.	Units
f_x, f_y (focal length)	W	320	640	1280	px
c_x (principal point x)	$\frac{W}{2}$	160	320	640	px
c_y (principal point y)	$\frac{H}{2}$	90	180	360	px
Distortion	`plumb_bob`	*N/A*	$k_i = 0$	*N/A*	*N/A*

IBVS Visual Error. The system employs Image-Based Visual Servoing (IBVS): the control error is computed directly in the image plane rather than in 3D Cartesian space. Given the YOLO-detected target centre (x_t, y_t) and the frame centre (c_x, c_y):

$$e_{x(t)} = x_t - c_x \tag{2}$$

A positive e_x indicates the target lies to the right of the optical axis. This scalar is fed as the process variable to the lateral PID controller (see Section 3).

Nearest-Target Depth Proxy. When multiple tennis balls appear in a single frame, the system selects the nearest candidate using the bottom Y-coordinate heuristic:

$$\text{target} = \arg\max_{\text{box}_i}(y_{2,i}) \tag{3}$$

Larger y_2 values correspond to objects lower in the frame, which under the ground-facing camera assumption implies closer proximity.

1.2. YOLO Object Detection Theory

Architecture. The system deploys YOLOv6-Nano (`yolo26n.pt`), a single-stage anchor-free detector comprising three components:

- **Backbone:** EfficientRep — a re-parameterisation-friendly CNN for multi-scale feature extraction.
- **Neck:** Rep-PAN — a Path Aggregation Network with re-parameterisation blocks.
- **Head:** Decoupled head with independent classification and localisation branches.

Loss Functions. Training minimises the composite loss:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}}\mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{DFL}} \tag{4}$$

where:

- $\mathcal{L}_{\text{CIoU}}$ (Complete IoU loss) regresses bounding boxes by jointly optimising overlap area, centre-point distance, and aspect ratio consistency.
- $\mathcal{L}_{\text{BCE}}$ (Binary Cross-Entropy) penalises class misclassification.
- $\mathcal{L}_{\text{DFL}}$ (Distribution Focal Loss) models box edge positions as discrete probability distributions for anchor-free regression.
- $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, $\lambda_{\text{dfl}} = 1.5$ are empirically weighted.

Training Hyperparameters.

Table 2: YOLO Training Hyperparameters

Hyperparameter	Value	Min.	Typical
Epochs	200	50	200–300
Input size (imgsz)	640	320	640
Batch size	16	4	16–64
Initial LR ($lr0$)	0.01	0.001	0.01
Final LR factor (lrf)	0.01	0.001	0.01
Optimiser	SGD	*N/A*	SGD (momentum = 0.937)
Weight decay	0.0005	0	0.0005
Warmup epochs	3.0	0	3.0
Mosaic augmentation	1.0	0	1.0
Horizontal flip probability	0.5	0	0.5
HSV perturbation	(0.015, 0.7, 0.4)	*N/A*	*N/A*
Random erasing probability	0.4	0	0.4
Early-stopping patience	100	10	50–100
Automatic Mixed Precision (AMP)	ON	*N/A*	ON

Inference Parameters.

Table 3: YOLO Inference Parameters

Parameter	Value	Min.	Typical
Confidence threshold	0.25	0.05	0.25–0.50
NMS IoU threshold	0.7	0.3	0.5–0.7
Inference resolution	640×360	*N/A*	*N/A*
Input channels	3 (RGB)	*N/A*	*N/A*

Model Conversion Pipeline. For edge deployment on the NVIDIA Jetson Orin NX, the model undergoes a three-stage optimisation:

$$\text{PyTorch (.pt)} \xrightarrow{\text{export}} \text{ONNX (.onnx)} \xrightarrow{\text{trtexec}} \text{TensorRT (.engine, FP16)} \tag{5}$$

The FP16 half-precision engine reduces inference latency by approximately $2\times$ with negligible detection accuracy degradation on the Jetson's Ampere GPU.

1.3. LiDAR Triangulation Principle

The YDLIDAR X3-YB-1 operates on the laser triangulation principle. An infrared laser emitter projects a spot onto the target surface; the reflected spot is imaged by a CMOS sensor at a known baseline offset from the emitter. The

distance d to the target is recovered by solving the triangle formed by the laser axis, the lens optical axis, and the reflected ray path:

$$d = \frac{b \cdot f}{\Delta x} \tag{6}$$

where b is the baseline distance, f the receiver lens focal length, and Δx the spot displacement on the CMOS array. The sensor head rotates at 7 Hz to achieve 360° scanning coverage.

Table 4: LiDAR Operating Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
Sampling rate	*N/A*	4000	*N/A*	Hz	Points per second
Scan frequency	*N/A*	7	*N/A*	Hz	Motor rotation
Angular resolution	0.6	*N/A*	*N/A*	°	At 7 Hz
Range	0.05	*N/A*	10	m	Indoor, 80% reflectivity
Minimum filtering threshold	*N/A*	20	*N/A*	mm	`MIN_DIST_MM`

1.4. IR Proximity Sensing

The RoboMaster EP chassis provides a built-in infrared proximity sensor. Range data $d_{\mathrm{IR}(t)}$ is polled at 10 Hz via the SDK subscription callback and used as a continuous scalar input (unit: millimetres) for longitudinal PID control and ingestion stage triggering.

Table 5: IR Distance Sensor Parameters

Parameter	Min.	Typical	Max.	Units
Polling frequency	*N/A*	10	*N/A*	Hz
Valid range (operational)	10	*N/A*	8848	mm
Pre-ingestion threshold	*N/A*	200	*N/A*	mm
Blind-zone range	*N/A*	< 450	*N/A*	mm
Ingestion-confirmation threshold	*N/A*	> 250	*N/A*	mm
Sentinel (uninitialised)	*N/A*	8848	*N/A*	mm

The sentinel value 8848 mm (height of Mount Everest in metres) is used to represent an infinitely far reading when the sensor has not yet been initialised, causing the controller to default to a pure vision-guided mode.

2. Mecanum Wheel Kinematics

2.1. 3-DOF Full Holonomic Inverse Kinematics (Nav2 Mode)

For autonomous navigation with Nav2, the Twist velocity command $\boldsymbol{v} = (v_x, v_y, \omega_z)$ is decomposed into per-wheel speeds using the standard Mecanum inverse kinematics:

$$\begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{pmatrix} = \gamma \cdot \begin{pmatrix} 1 & 1 & 1 \\ 1 & -1 & -1 \\ 1 & -1 & -1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ \omega_z \end{pmatrix} \tag{7}$$

where $\gamma = 100$ is an empirical RPM scaling factor. Explicitly:

$$\begin{aligned} \omega_{\text{right}} &= v_x + v_y + \omega_z \\ \omega_{\text{left}} &= v_x - v_y - \omega_z \end{aligned} \tag{8}$$

with the final wheel assignments:

$$\begin{aligned} \text{w1 (right-front)} &= \omega_{\text{right}} \times 100 \\ \text{w2 (left-front)} &= \omega_{\text{left}} \times 100 \\ \text{w3 (left-rear)} &= \omega_{\text{left}} \times 100 \\ \text{w4 (right-rear)} &= \omega_{\text{right}} \times 100 \end{aligned} \tag{9}$$

2.2. 2-DOF Simplified Inverse Kinematics (Reactive Mode)

For visual servoing, the control is reduced to two scalar commands — forward speed v_f and turn speed v_t:

$$\begin{aligned} \omega_{\text{left}} &= v_f + v_t \\ \omega_{\text{right}} &= v_f - v_t \end{aligned} \tag{10}$$

This formulation sacrifices lateral strafing capability but allows the dual-PID controller to directly govern approach-while-aligning behaviour with two degrees of freedom.

2.3. Primitive Motion Analysis

Table 6: Wheel-Speed Decomposition for Fundamental Motions (2-DOF Model)

Motion	ω_{left}	ω_{right}
Pure forward ($v_t = 0$)	v_f	v_f
Pure rotation ($v_f = 0$)	$+v_t$	$-v_t$

Wheel Numbering Convention. The DJI SDK `drive_wheels(w1, w2, w3, w4)` interface adopts the following mapping:

Table 7: DJI SDK Wheel Convention

w1	w2	w3	w4
Right-Front	Left-Front	Left-Rear	Right-Rear

3. PID Control Theory

The reactive chassis controller employs two independent PID regulators in the standard parallel form:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\,\mathrm{d}\tau + K_d \frac{\mathrm{d}e(t)}{\mathrm{d}t} \tag{11}$$

where $e(t)$ is the control error at time t, and $\left(K_p, K_i, K_d\right)$ are the proportional, integral, and derivative gains respectively.

3.1. Lateral Alignment PID (`pid_turn`)

This controller drives the target's image-plane centroid offset e_x (Equation 2) to zero, aligning the robot with the ball.

Table 8: Lateral PID Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
K_p	*N/A*	0.15	*N/A*	RPM/px	Proportional gain
K_i	*N/A*	0.03	*N/A*	RPM/(px dot s)	Integral gain
K_d	*N/A*	0.03	*N/A*	RPM dot s/px	Derivative gain
Setpoint	*N/A*	0	*N/A*	px	Centred on optical axis
Output lower	−50	*N/A*	*N/A*	RPM	Max left-turn speed
Output upper	*N/A*	*N/A*	+50	RPM	Max right-turn speed

The output is negated before application: a target on the right ($e_x > 0$) produces a negative turn command, i.e., clockwise rotation.

3.2. Longitudinal Approach PID (`pid_forward`)

This controller regulates the IR-measured distance $d(t)$ toward a 10 mm setpoint (physical contact with the ball).

Table 9: Longitudinal PID Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
K_p	*N/A*	0.5	*N/A*	RPM/mm	Proportional gain
K_i	*N/A*	0.1	*N/A*	RPM/(mm dot s)	Integral gain
K_d	*N/A*	0.05	*N/A*	RPM dot s/mm	Derivative gain
Setpoint	*N/A*	10	*N/A*	mm	Ball contact distance
Output lower	−40	*N/A*	*N/A*	RPM	Max reverse speed
Output upper	*N/A*	*N/A*	+40	RPM	Max forward speed

3.3. Dual-PID Visual Servoing Architecture

The two controllers operate in a priority-ordered cascade:

1. **Cruise tracking** ($d > 800$ mm): Both PID loops active. If $|e_x| > 40$ px, forward speed is clamped to 5 RPM to prioritise lateral alignment over advance.
2. **PID approach** ($d \leq 800$ mm): The `pid_forward` output overrides the open-loop cruise speed.
3. **Blind-zone relay**: When visual tracking is lost (e_x unavailable) but both $d < 450$ mm and last-valid distance $d_{\text{last}} < 300$ mm, the chassis maintains a straight 30 RPM advance with zero turn command. This bridges the camera forward-mount dead zone.
4. **Ingestion trigger** ($d \leq 200$ mm): Motors pre-spin, chassis advances at fixed 30 RPM. The system transitions to the ingestion confirmation window.

The blind-zone relay condition is formalised as:

$$\neg\ \text{target\_valid} \wedge (d < 450) \wedge (d_{\text{last}} < 300) \tag{12}$$

Ingestion Confirmation. Ingestion is confirmed via a sliding-window majority vote on the distance sensor:

$$\text{is\_swallowed} = \left(\sum_{k=1}^{10} \mathbf{1}[d(t + k \cdot 20\ \text{ms}) > 250] \right) \geq 10 \tag{13}$$

i.e., 10 consecutive readings exceeding 250 mm within a 200 ms window, protected by a 3-second hard timeout.

4. Navigation & Planning Theory

4.1. SLAM (`slam_toolbox`)

The system employs `slam_toolbox` in Online Asynchronous mode for Simultaneous Localisation and Mapping. The algorithm is built on a sparse pose-graph:

- **Nodes** represent robot poses (x_i, y_i, θ_i) at key scan timestamps.
- **Edges** encode spatial constraints: odometry edges (from wheel odometry) and scan-matching edges (from LiDAR scan alignment).
- **Scan matching** uses Karto-style correlative scan matchers (multi-resolution search over (x, y, θ)) to align incoming LiDAR scans against the accumulated occupancy grid.
- **Global optimisation**: The pose-graph is periodically optimised via non-linear least squares, minimising:

$$\boldsymbol{X}^* = \arg\min_{\boldsymbol{X}} \sum_{i,j} \boldsymbol{e}_{ij}^T \boldsymbol{\Omega}_{ij} \boldsymbol{e}_{ij} \tag{14}$$

  where $\boldsymbol{e}_{ij}$ is the constraint error between nodes i and j and $\boldsymbol{\Omega}_{ij}$ is the information matrix.
- **Loop closure**: Upon revisiting a previously mapped region, additional scan-matching constraints are added and the full graph is re-optimised.

The resulting map is a 2D occupancy grid, where each cell stores the log-odds $l_{x,y}$ of being occupied:

$$p(\text{occupied} \mid z_{1:t}) = 1 - \frac{1}{1 + \exp(l_{x,y})} \tag{15}$$

4.2. AMCL Localisation

Adaptive Monte Carlo Localisation (AMCL) estimates the robot pose $\boldsymbol{x}_t = (x, y, \theta)_t$ through a particle filter:

1. **Prediction**: Each particle is propagated by sampling from the motion model $p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}, \boldsymbol{u}_{t-1})$ using wheel odometry.
2. **Update**: Each particle's weight is updated by the measurement model $p(\boldsymbol{z}_t \mid \boldsymbol{x}_t, m)$ using LiDAR scan likelihood against the pre-built map m.
3. **Resampling**: Particles are resampled proportionally to weights (KLD-sampling for adaptive particle count).

4.3. Smac Hybrid-A* Global Planner

Nav2 employs the Smac Hybrid-A* planner, which extends classical A* search to a continuous state space. The hybrid state $\boldsymbol{s} = (x, y, \theta)$ is evaluated with:

$$f(\boldsymbol{s}) = g(\boldsymbol{s}) + h(\boldsymbol{s}) \tag{16}$$

where $g(\boldsymbol{s})$ is the accumulated cost-to-come (including penalties for proximity to obstacles and non-smooth transitions) and $h(\boldsymbol{s})$ is an admissible heuristic (typically a 2D Euclidean distance with obstacle-awareness). The continuous-state feasibility check ensures that generated states satisfy the robot's kinematic constraints (minimum turning radius).

4.4. Regulated Pure Pursuit Local Controller

The Regulated Pure Pursuit (RPP) controller computes angular velocity ω from a look-ahead point (x_l, y_l) on the global path:

$$\omega = \frac{2v \sin(\alpha)}{L_d} \tag{17}$$

where v is the current linear velocity, α is the angle between the robot's heading and the look-ahead point, and L_d is the adaptive look-ahead distance. The regulation heuristics include velocity scaling near the goal, collision checking on the look-ahead arc, and curvature-based deceleration.

4.5. Boustrophedon Coverage Path

The workspace coverage strategy generates an S-shaped (boustrophedon) path through all traversable grid cells. Given a calibrated workspace $\mathcal{W}$ discretised into cells of dimension $\Delta x \times \Delta y$ (matching the intake mechanism width), the path comprises alternating horizontal sweeps:

$$\mathcal{P}_{\text{coverage}} = \{\boldsymbol{w}_k \in \mathcal{W}_{\text{traversable}} \mid k = 1, \ldots, n\} \tag{18}$$

Grid cells are classified into one of four states:

Table 10: Grid Cell State Classification

State	Symbol	Semantics
Occupied	■	Static obstacle (unreachable)
Traversable	□	Free space, not yet visited
Cleared	✓	Cell has been swept, no ball remaining
Target	•	Ball detected via YOLO in this cell

5. Finite State Machine

The behavioural logic of the CourtSweeper is formally defined as a Mealy-type Finite State Machine (FSM).

Formal Definition.

$$\mathcal{M} = (Q, \Sigma, \delta, q_0, F) \tag{19}$$

where:

- $Q = \{q_0, q_1, q_2, q_3\}$ is the finite set of states.
- Σ is the input alphabet of events (perception events, sensor thresholds, timer expirations, and UI commands).
- $\delta : Q \times \Sigma \to Q$ is the transition function.
- $q_0 = \text{IDLE}$ is the initial state.
- $F = \emptyset$ (no terminal states; the FSM runs indefinitely).

5.1. State Definitions

Table 11: FSM State Set

ID	State	Symbol	Semantics
0	IDLE	q_0	Awaiting command. Manual teleoperation overrides autonomy. Court calibration performed here.
1	SEARCHING	q_1	No ball detected. Chassis follows the pre-computed boustrophedon coverage path, marking traversed cells as cleared.
2	TRACKING	q_2	Ball detected. Coverage path paused. Dual-PID controller aligns chassis with target and navigates toward projected map coordinates.
3	INGESTING	q_3	Ball within 200 mm. Roller motors pre-spin; chassis advances; sliding-window detector confirms ingestion.

5.2. Event Alphabet

The FSM responds to the following events:

Table 12: FSM Input Events (Σ)

Event	Symbol	Trigger Condition
Ball detected	e_{detect}	`target_locked = True` (YOLO confidence ≥ 0.25)
Ball lost	e_{lost}	`no_ball_counter >= 15` (consecutive frames)
Approach threshold	e_{approach}	$d_{\text{IR}} \leq 200$ mm
Ball ingested	e_{ingested}	`is_swallowed = True` (Equation 13)
Ingestion timeout	e_{timeout}	Elapsed 3 s without ingestion confirmation
Coverage complete	e_{done}	All traversable cells marked cleared
Auto mode	e_{auto}	UI AUTO switch toggled
Manual override	e_{manual}	UI MANUAL switch toggled or teleop input received
E-STOP	e_{estop}	UI E-STOP button pressed

5.3. Transition Function

Table 13: FSM Transition Table $\delta : Q \times \Sigma \to Q$

From	Event	To
IDLE	e_{auto}	SEARCHING
IDLE	e_{detect}	TRACKING
SEARCHING	e_{detect}	TRACKING
SEARCHING	e_{done}	IDLE
TRACKING	e_{lost}	SEARCHING
TRACKING	e_{approach}	INGESTING
INGESTING	e_{ingested}	SEARCHING
INGESTING	e_{timeout}	SEARCHING
Any	e_{manual}	IDLE
Any	e_{estop}	IDLE

6. Coordinate Transformations

6.1. Euler Angle to Quaternion

The robot operates on a planar surface; yaw θ is the only non-zero Euler angle. The conversion to quaternion representation follows the axis-angle formula for a Z-axis rotation:

$$\boldsymbol{q}(\theta) = (q_x, q_y, q_z, q_w) = \left(0, 0, \sin\frac{\theta}{2}, \cos\frac{\theta}{2}\right) \tag{20}$$

6.2. Yaw Normalisation

To prevent numerical drift due to θ wrapping outside $[-\pi, \pi]$, the yaw is normalised using:

$$\theta' = \text{atan2}(\sin\theta, \cos\theta) \tag{21}$$

6.3. TF Coordinate Tree

The ROS2 TF tree defines the rigid transformations in Table 14.

Table 14: TF Frame Hierarchy

Parent Frame	Child Frame	Δx	Δy	Δz	$\Delta\theta$	Remarks
`odom`	`base_footprint`	SLAM estimate	SLAM estimate	0	SLAM estimate	AMCL-provided
`base_footprint`	`base_link`	0	0	0	0	Static (coincident origins)
`base_link`	`laser`	0	0	0	0	Static (LiDAR mount)

Coordinate Sign Convention.

- **Mapping mode** (`map_generation.py`): Yaw and x-coordinate are negated ($\theta \to -\theta$, $x \to -x$) to align the DJI odometry frame with the SLAM map convention.
- **Navigation mode** (`map_navigation.py`): Raw DJI coordinates are preserved; the saved map already conforms to the localisation frame.

7. Communication Protocol Specifications

7.1. ROS2 Inter-Process Communication

Table 15: ROS2 Topic Specification

Topic	Message Type	Freq. (Hz)	Direction	Remarks
`/odom`	`nav_msgs/Odometry`	50	Chassis → ROS2	DJI SDK telemetry
`/scan`	`sensor_msgs/LaserScan`	≃ 10	LiDAR → ROS2	RPLidar driver
`/cmd_vel`	`geometry_msgs/Twist`	On demand	ROS2 → Chassis	Nav2 output
`/camera/image/compressed`	`sensor_msgs/CompressedImage`	10	Chassis → ROS2	JPEG, quality 60
`/camera/camera_info`	`sensor_msgs/CameraInfo`	10	Chassis → ROS2	Intrinsic calibration
`/tf`	`tf2_msgs/TFMessage`	50	Bridge → ROS2	Coordinate frames

7.2. UDP Control Protocol: iOS to Jetson

Table 16: UDP Command Packet Specification

Command	Payload Format	Semantics
E-STOP	`{"cmd": "estop"}`	Immediate chassis brake, suspend all autonomous logic
RECALL	`{"cmd": "recall"}`	Terminate mission, navigate to origin via Nav2
AUTO	`{"cmd": "auto"}`	Switch FSM to autonomous mode
MANUAL	`{"cmd": "manual"}`	Switch FSM to manual teleoperation
JOYSTICK	`{"vx": f, "vy": f, "vw": f}`	Omnidirectional velocity command

Table 17: UDP Transport Parameters

Parameter	Min.	Typical	Units
Serialisation	*N/A*	JSON	*N/A*
Packet size (commands)	30	80	bytes
Target latency	5	15	ms

7.3. Serial Motor Protocol: Jetson to Arduino

Table 18: Serial Communication Parameters

Parameter	Min.	Typical	Max.	Units	Remarks
Baud rate	*N/A*	115200	*N/A*	bps	8N1 (8 data, 1 stop, no parity)
Packet format	*N/A*	`"L,R\n"`	*N/A*	*N/A*	CSV-style, newline-terminated
Speed range	−255	*N/A*	+255	*N/A*	8-bit signed PWM
Encoding	*N/A*	UTF-8	*N/A*	*N/A*	*N/A*

7.4. Wi-Fi Connection Modes

Table 19: Wi-Fi Operation Modes

Mode	SDK Parameter	Use Case
Access Point (AP)	`conn_type="ap"`	Direct iOS connection; Jetson broadcasts SSID for dataset collection and field teleop
Station (STA)	`conn_type="sta"`	Robot/Jetson joins existing Wi-Fi network for SLAM mapping and Nav2 navigation

8. Motor Drive Theory

8.1. PWM Duty Cycle Control

The Arduino firmware generates Pulse Width Modulation signals with 8-bit resolution. The duty cycle is:

$$D = \frac{|\text{speed}|}{255} \times 100\% \tag{22}$$

where speed $\in [-255, 255]$ is the signed command. The effective motor voltage is:

$$V_{\text{motor}} = D \cdot V_{\text{supply}} = D \cdot 11.1 \text{ V} \tag{23}$$

Pre-spin duty (speed $= 90$) corresponds to $D \simeq 35\%$, yielding approximately 3.9 V across the 775 motor terminals.

8.2. BTS7960 H-Bridge Topology

Each BTS7960 driver module forms a full H-bridge using two half-bridge outputs. The three-wire interface operates as follows:

Table 20: BTS7960 Logic Truth Table

EN	RPWM	LPWM	Motor State	Condition
LOW	0	0	Coast (free-running)	speed $= 0$
HIGH	$\lvert s \rvert$	0	Forward rotation	speed > 0
HIGH	0	$\lvert s \rvert$	Reverse rotation	speed < 0

The enable pin (EN) controls the H-bridge output stage: pulling it LOW disconnects both half-bridges, providing a passive coast-stop that prevents back-EMF-induced braking torque.

Pin Mapping.

Table 21: Arduino Pin Assignment for Dual BTS7960 Drivers

Motor	EN	RPWM	LPWM
Left	D4	D5	D6
Right	D7	D9	D10

8.3. Communication Watchdog

A hardware-level safety mechanism is implemented in firmware:

$$\text{if } (\text{isMoving} \wedge (t_{\text{now}} - t_{\text{lastRecv}} > 500 \text{ ms})) \Rightarrow \text{stopMotor}() \tag{24}$$

If no serial command is received within 500 ms while motors are in motion, all PWM channels are zeroed and the enable pins are pulled LOW. This prevents runaway behaviour in the event of communication loss (e.g., serial cable disconnection, Jetson crash, or software deadlock).

Command Parsing. The Arduino firmware uses `Serial.parseInt()` to extract signed integers from the CSV stream, validating each packet with a newline terminator check (`Serial.read() == '\n'`). Incomplete or malformed packets are silently discarded, and the last valid speed values are retained.